

Rapport de projet — Spacegame (LU3IN032)

par Alexandre Hurel et Phivos Koulmasis

1. Manuel d'utilisation

Prérequis

- **GHC + Stack** installés (via [GHCup](#) sur Windows ou Linux).
- Sous Linux (Ubuntu/Debian), les bibliothèques OpenGL/GLUT sont nécessaires pour la version graphique :

```
sudo apt install -y libgl1-mesa-dev libglu1-mesa-dev freeglut3-dev
```

Compiler

Depuis la racine du dossier `spacegame/` :

```
stack build
```

Lancer

Version graphique :

```
stack run spacegame-gloss
```

Version terminal :

```
stack run spacegame-exe
```

Avec un fichier d'obstacles initiaux :

```
stack run spacegame-gloss -- chemin/vers/obstacles.txt
```

Le fichier obstacles est un fichier ASCII où # marque la position d'un obstacle sur la grille 30×20.

Contrôles

Touche	Action
Z,Q,S,D	Déplacements
Espace	Tirer

Objectif

Accumuler du score en détruisant ennemis et obstacles. Lorsque le score atteint le seuil du niveau courant ($500 \times n \times (n+1)/2$), le niveau suivant démarre. Le boss apparaît en fin de niveau, le vaincre est requis pour progresser.

Lancer les tests

```
stack test
```

2. Bilan de développement

Fonctionnalités implémentées

Fonctionnalité	Description
Déplacement du joueur	4 directions, bornes de l'écran respectées
Tir du joueur	Projectile montant, maintien de touche en version Gloss
Trois types d'ennemis	Direct, Zigzag, Tireur (comportements distincts)
Projectiles ennemis	Zigzag (1/4 chance), Tireur (chaque tour)
Obstacles défilants	Scrollent vers le bas, détruits par les lasers

Fonctionnalité	Description
Boss de fin de niveau	Rebond latéral, tir en éventail variable selon le niveau
Système de bonus	ExtraVie (+1 PV), BonusScore (+200 pts)
Difficulté progressive	<code>configPourNiveau</code> paramètre fréquences et PV boss
Niveaux infinis	Score quadratique, transitions avec compte à rebours
Interface graphique Gloss	Sprites procéduraux, interpolation, HUD pixel-art
Interface terminale ASCII	Lecture non-bloquante, affichage ANSI
Chargement de fichier niveau	Parser ASCII via <code>chargerObstacles</code>

Extensions choisies

1. **Interface graphique complète (Gloss)** : rendu avec polygones procéduraux ou sprites PNG, interpolation bilinéaire entre les ticks, HUD avec cœurs pixel-art, transitions entre niveaux.
2. **Trois types d'ennemis différents** : comportement de déplacement et de tir distincts, difficulté croissante dans les niveaux.
3. **Boss avec patterns de tir évolutifs** : le boss adapte son nombre de projectiles (1, 3, 5) et sa cadence de tir selon le niveau courant, rendant les niveaux supérieurs nettement plus difficiles.
4. **Difficulté paramétrable par niveau** (`ConfigNiveau`) : séparation propre entre paramètres et logique de jeu, permet d'ajuster la difficulté indépendamment pour chaque niveau.

Propriétés et tests

La suite de tests couvre 28 cas :

- `charToDir` : 5 tests (4 directions + cas générique)
- `depJ` : 4 tests de bord (gauche, haut, droite, bas) + 2 propriétés QuickCheck
- `scrollObstacles` : 3 tests dont 1 propriété QuickCheck
- `impactProj` : 3 tests (détection, case vide, case joueur non-cible)
- `valeurBonus` : 3 tests (BonusScore, ExtraVie, Nothing)
- `configPourNiveau` : 5 tests (PV boss niveaux 1/3/5, fréquence ennemis)
- `GlossCoords.toGlossXY` : 3 tests (coins et centre de grille)

Propriétés QuickCheck :

- `prop_depJ_col_inBounds` : $\forall$ direction, la colonne reste dans `[0, largeur-1]`
- `prop_depJ_lig_inBounds` : $\forall$ direction, la ligne reste dans `[0, hauteur-1]`

- `prop_scroll_inBounds` : après `scrollObstacles` , aucun obstacle hors écran

3. Rapport de développement

Architecture générale

Le projet est découpé en trois couches clairement séparées :

- `src/Lib.hs` : logique de jeu entièrement pure (sans IO). Contient tous les types, les trois monades obligatoires, et toutes les fonctions de gameplay.
- `app-gloss/` : couche graphique (Gloss). `GlossUI.hs` gère le rendu et les événements ; `GlossCoords.hs` la conversion de coordonnées.
- `app/Main.hs` : couche IO terminale. Boucle de jeu avec lecture clavier non-bloquante.

Cette architecture garantit que toute la logique est testable sans IO ni dépendance graphique.

Monade État

```
newtype Etat s a = Etat { runEtat :: s -> (a, s) }

instance Functor (Etat s) where
  fmap f (Etat g) = Etat $ \s -> let (a, s') = g s in (f a, s')

instance Applicative (Etat s) where
  pure a           = Etat $ \s -> (a, s)
  Etat ef <*> Etat ex = Etat $ \s ->
    let (f, s') = ef s ; (a, s'') = ex s' in (f a, s'')

instance Monad (Etat s) where
  Etat ex >>= f = Etat $ \s ->
    let (a, s') = ex s ; Etat eg = f a in eg s'
```

Les instances sont implémentées à la main, sans recourir à MTL ou transformers. Le type `EtatJeu a = Etat Envi a` sert pour toutes les actions de gameplay. L'état `Envi` est passé implicitement par la `do`-notation, ce qui évite des signatures surchargées.

La fonction `runJeu :: EtatJeu a -> Envi -> (a, Envi)` est le pont vers IO : elle extrait le résultat et le nouvel état sans rester dans la monade.

Monade Maybe et Data.Map

Deux usages de la monade Maybe illustrent le chaînage de lookups :

valeurBonus — deux lookups dépendants :

```
valeurBonus c bonusMap effetMap = do
  b      <- Map.lookup c bonusMap      -- bonus présent à cette case ?
  valeur <- Map.lookup (bonusType b) effetMap -- valeur du type de bonus ?
  return (bonusType b, valeur)
```

Le second lookup dépend du résultat du premier (`bonusType b`). Si l'un échoue, `Nothing` se propage.

impactProj — lookup puis filtrage dans Maybe :

```
impactProj c grille = do
  cellules <- Map.lookup c grille
  let cibles = filter estCible cellules
  if null cibles then Nothing else Just cibles
```

La grille `Map Coord [Cellule]` est construite via `Map.fromListWith (++)`, permettant plusieurs entités par case et des requêtes en $O(\log n)$.

Correction du tunneling laser/obstacle

Problème découvert : dans la fonction `tour`, les projectiles montent (`avancerProjectiles : lig - 1`) avant que les obstacles descendent (`scrollObstacles : lig + 1`). Quand un laser est adjacent à un obstacle en dessous de lui, ils échangent leurs positions en un seul tick sans jamais occuper la même case — `traiterCollisionsProj` ne détecte aucune collision.

Exemple concret :

```
Avant tick :  laser en (c, 5),   obstacle en (c, 4)
Après tick  :  laser en (c, 4),   obstacle en (c, 5)
Grille :      laser ≠ obstacle → aucune collision détectée !
```

Correction : dans `traiterCollisionsProj`, après la détection classique (positions identiques), on ajoute une détection de croisement : pour un laser en `(c, 1)`, on vérifie si une cible se trouve en `(c, 1+1)` dans la grille courante (position qu'elle occupait avant de descendre).

```

-- Hits classiques (même case)
hitsPos = [ projCoord p | p <- projSJ, isJust (impactProj (projCoord p) grille) ]

-- Hits par croisement : laser en (c,l) + cible en (c,l+1) → tunneling
crossPairs = [ (projCoord p, Coord c (l + 1))
               | p <- projSJ
               , let Coord c l = projCoord p
               , isJust (impactProj (Coord c (l + 1)) grille) ]
crossLasers = map fst crossPairs -- lasers à consommer
crossTargets = map snd crossPairs -- cibles à endommager

```

Les lasers des deux listes sont consommés ; les cibles (obstacles ou ennemis) sont endommagées/détruites selon leur type. Cette correction s'applique aux obstacles et aux ennemis, qui descendent tous d'une ligne par tick.

Correction de l'invulnérabilité du boss

Problème découvert : deux bugs indépendants rendaient le boss invulnérable aux lasers du joueur.

Bug 1 — consommation prématurée des lasers. La fonction `traiterCollisionsProj` utilisait initialement `impactProj`, qui inclut `CellBoss` parmi les cibles. Ainsi, tout laser touchant la case du boss était consommé par `traiterCollisionsProj` avant que `traiterCollisionsBoss` ne s'exécute. Résultat : `traiterCollisionsBoss` ne voyait aucun laser à traiter, et le boss ne perdait jamais de PV.

Correction du Bug 1 : remplacement de `impactProj` par une version locale `impactOE` (obstacles + ennemis uniquement, `CellBoss` explicitement exclu) dans `traiterCollisionsProj`. Les lasers ciblant le boss ne sont plus consommés prématurément.

Bug 2 — ordre des appels dans `tour`. Même après le Bug 1 corrigé, le boss semblait toujours invulnérable. La cause : dans `tour`, `bougerBoss` était appelé avant `traiterCollisionsBoss`. Le boss se déplace latéralement d'une case par tick ; si un laser arrive en `(c, 1)` et que le boss est en `(c, 1)`, `bougerBoss` le déplace en `(c+1, 1)` avant la vérification de collision. `traiterCollisionsBoss` compare alors `projCoord p == bossCoord b` soit `(c, 1) ≠ (c+1, 1) → aucun hit détecté`.

```

Avant tick : laser en (c, 1), boss en (c, 1)
Après bougerBoss : boss en (c+1, 1)
traiterCollisionsBoss : (c,1) ≠ (c+1,1) → pas de dégât !

```

Correction du Bug 2 : `traiterCollisionsBoss` est maintenant appelé juste après `bougerEnnemis` et juste avant `bougerBoss`. Les lasers sont comparés à la position actuelle du boss, avant tout

déplacement de ce dernier.

Difficulté progressive

`configPourNiveau :: Int -> ConfigNiveau` calcule pour chaque niveau :

```
configPourNiveau n = ConfigNiveau
  { cfgFreqObs    = max 1 (12 - n)  -- niv.1 : 1/11 ticks ≈ 0.7/s
  , cfgFreqEn     = max 1 (9 - n)   -- niv.1 : 1/8 ticks = 1/s
  , cfgPVBoss     = 3 * n + 2       -- niv.1 : 5PV → niv.5 : 17PV
  , cfgScoreBoss  = max 100 (scoreRequis n - 300)
  }
```

Le score requis suit une croissance quadratique : $\text{scoreRequis } n = 500 \times n \times (n+1) / 2$ (500, 1500, 3000...), assurant que chaque niveau est nettement plus long que le précédent.

Interface graphique Gloss

Sprites procéduraux : si les PNG sont absents, les sprites sont générés via des polygones Gloss (`ngon`, `starPicture`, etc.). La fonction `loadOrGen` tente le chargement PNG et utilise le sprite procédural comme fallback.

Interpolation bilinéaire: à 60 FPS d'affichage pour 8 ticks/s de logique, un facteur

$t = \text{accum} / \text{tickInterval} \in [0, 1]$ interpole la position des entités entre deux ticks consécutifs pour un rendu fluide.

Gestion du tir maintenu : `gFireHeld` permet de tirer en continu en maintenant Espace, tandis que `gFire` gère un tir unitaire par appui.

Difficultés rencontrées

- **Lecture clavier non-bloquante sous Windows** : la bibliothèque standard Haskell n'offre pas `hReady` sous Windows. Solution : import FFI des fonctions C `_kbhit` et `_getch` via `foreign import ccall`.
- **Tunneling laser/obstacle** : bug subtil dû à l'ordre des opérations dans `tour`. Détecté en observant que certains lasers tirés à bout portant traversaient les obstacles sans les détruire. Corrigé par la détection de croisement décrite ci-dessus.
- **Coordination Gloss/logique** : Gloss impose un modèle à callbacks purs (`play`). L'accumulation de temps (`gAccum`) et la séparation nette entre `GlossEnv` et `Env` ont été nécessaires pour découpler le rendu (60 FPS) de la logique (8 ticks/s).

4. Bilan des sources

Apprentissage du langage

Regarder cette vidéo pendant les vacances nous a aidé à nous replonger dans Haskell :

https://youtu.be/TkIkNLihQ_A?si=GscTpl8-4PIRxZpm

Nous avons aussi développé une série de tests en nous aidant de cette page :

<https://stackoverflow.com/questions/7751256/how-to-test-my-haskell-functions>

Utilisation de Claude

Nous avons utilisé Claude pour nous débloquer de situations où nous allions autrement passer des heures à comprendre/résoudre. Voici quelques prompts :

- Real Time rendering :
[lien vers le github] Voici un repo avec un jeu en ASCII entièrement programmé en Haskell. Actuellement le jeu display le terminal et attend un input puis entrée afin de refresh la frame d'après, j'aimerais que tu modifies celà afin que le jeu soit en temps réel.
- Comprendre Gloss :
[lien vers le github] Je souhaiterais utiliser la bibliothèque Gloss afin de mettre une interface graphique au dessus de la représentation ASCII du jeu. Explique moi les fondamentaux de cette bibliothèque. Quelle serait une architecture Gloss adéquate ici ?
- Correction de notre première approche de Gloss :
[ancien fichier MainGloss.hs] Corrige moi ce fichier et explique moi les erreurs commises dans l'implémentation.
- Séparer fluidité rendu et rapidité de la logique :
Actuellement le jeu tourne à 8 fps, quand j'augmente à 60, c'est plus fluide mais la logique est nettement plus rapide et le jeu est injouable. J'aimerais que tu me proposes dans un autre fichier une implémentation en séparant vitesse de rendu et vitesse de logique : j'ai pensé à utiliser pour la logique un diviseur du nombre de FPS à l'affichage afin de faire un calcul toutes les x frames sans en sauter.
- Optimisation générale du code :
[lien vers le github] Voici un projet entièrement fait en Haskell. Le contenu est probablement écrit de manière non optimale, j'aimerais qu'en gardant ce que j'ai fait à la main, tu optimises chaque fonction quand il existe plus simple pour faire ce que j'ai fait.
- Debugger collision entre boss et lasers :
[fichier lib.hs] pourquoi le boss ne perd-il pas de vie lorsque touché par un laser ? Corriger cette

fonction, explique mon erreur en commentaire, s'assurer que l'affichage se met bien à jour (c'est soit un problème de logique, soit d'affichage, possiblement les deux).

- Logique de niveaux :

[lien vers le github] Actuellement ce jeu en Haskell n'a qu'un niveau, et se termine soit quand le joueur gagne ou perd. J'aimerais que tu réfléchisses à une logique de niveau, où la difficulté serait proportionnelle au niveau actuel, ainsi que le temps passé sur chaque niveau. Le niveau 1 serait simple, le 2 plus dur etc... J'aimerais un reset de la position ainsi qu'un écran de transition entre chaque niveau afin de comprendre clairement qu'on passe d'un niveau à l'autre.

Pour conclure sur l'utilisation de l'IA dans ce projet, nous avons avant tout commencé à la main afin de comprendre le projet ce la base de celui-ci, puis nous avons corrigé des bugs / compris comment ajouter des extensions de manières optimales grâce à l'IA. A chaque fois nous avons donné des directives à Claude au lieu de le laisser décider de l'implémentation.

Sprites

L'ensemble des éléments graphiques utilisés on soit été trouvé sur une bilbiothèque open source (kenney) de sprite 2D, soit développés à la main avec Gimp. Nous nous sommes fortement inspirés de jeux déjà existants dans le même style.